

Dynamic Programming

COMP4128 Programming Challenges

School of Computer Science and Engineering
UNSW Sydney

Term 2, 2026

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP
- 5 DP with Data Structures
- 6 More example problems

Dynamic Programming

Reminder: Algorithmic Complexity

What is dynamic programming?

2D DP

Exponential DP

DP with Data Structures

More example problems

- The running time of your solution is important!
- If you don't think about the time complexity of your algorithm before coding it up, sooner or later you'll end up wasting a lot of time on something something that's too slow.
 - This is especially tragic in exam environments.
- For simple code, analysing complexity can be as simple as multiplying together the bounds of nested for loops.
- For recursive solutions, a rough bound is $O(\text{time spent in recursive function} \times \text{number of recursion branches}^{\text{recursion depth}})$
- For DP, it usually comes down to carefully determining the number of subproblems and the average time taken for each of them using the recurrence.

- On most online judges (this applies to the problem sets), a rough guideline is 200 million operations per second.
 - Constant factors occasionally matter, e.g. if you have no recursion, or only tail-recursion, you might manage more operations than this.
 - If you do floating-point arithmetic, everything will be slow
- This means that for $n \leq 1,000,000$, an $O(n \log n)$ algorithm will probably run in time, but an $O(n^2)$ algorithm will definitely time out.
- Best way to get a gauge of an online judge's speed is to submit a simple for loop and compare the number of iterations it can do in 1 second to your local environment.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP
- 5 DP with Data Structures
- 6 More example problems

- We like greedy algorithms because they cut down the state space
- If a locally suboptimal choice can never contribute to the globally optimal solution, we don't have to expand nearly as many states
- But what if this doesn't work? Hill climbing etc
- We would like some way to explore many options at each stage, but efficiently - avoid repeating work

What is dynamic programming?

7

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Wikipedia: “a method for solving complex problems by breaking them down into simpler subproblems”
- If we can then keep recursively breaking down those simpler subproblems into even simpler problems until we reach a subproblem which is trivial to solve, we are done.
- This sounds a lot like Divide & Conquer...
- The key aspect of Dynamic Programming is **subproblem reuse**:
 - If we have a divide & conquer algorithm that regularly reuses the same subproblem when breaking apart different larger problems, it'd be an obvious improvement to save the answer to that subproblem instead of recalculating it.
- In a way, dynamic programming is *smart recursion*

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

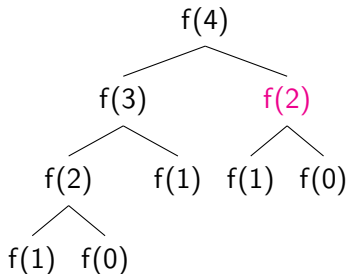
Exponential
DP

DP with Data
Structures

More example
problems

- **Problem statement** Compute the n th Fibonacci number ($0 \leq n \leq 1,000,000$)
- **Naïve solution** Recall that $f(0) = f(1) = 1$, and $f(n) = f(n-1) + f(n-2)$. Write a recursive function and evaluate.
- **Time Complexity** We recurse twice from each call to f , and the recursion depth is up to n . This gives a complexity of $O(2^n)$.

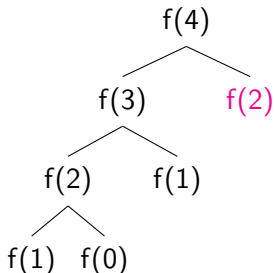
- Let's take a look at the call tree for $f(4)$:



- What is $f(2)$?

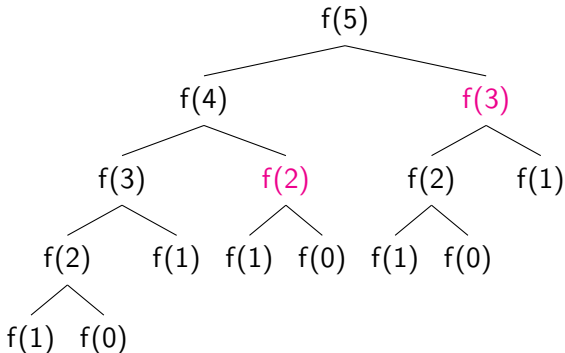
A problem we've seen before.

- If we don't duplicate work:

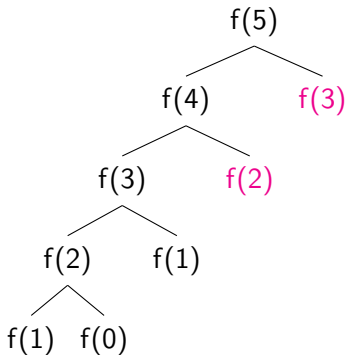


- The call tree gets a bit smaller.

- There is even more duplicated work in bigger cases.



- With “smart recursion” we could reduce the call tree to:



- In fact, we reduce the number of calls from $O(2^n)$ to $O(n)$.

In general, a dynamic programming (DP) algorithm comes in three parts:

- An exact definition of the subproblems. It is convenient to define these subproblems as entities in a *state space* and refer to individual subproblems as **states**.
 - In our example, each $f(i)$ is a state, and the state space includes all these states for i from 0 to n .
- A **recurrence relation**, which facilitates the breaking down of subproblems. These define the *transitions* between the states.
 - In our example, the recurrence relation is $f(n) = f(n-1) + f(n-2)$.
- **Base cases**, which are the trivial subproblems.
 - In our example, the base cases are $f(0) = 1$ and $f(1) = 1$.

- **Problem statement** Given an integer n ($0 \leq n \leq 1,000,000$), in how many ways can n be written as a sum of the integers 1, 3 and 4?
- **Example** If $n = 5$, there are 6 different ways:

$$\begin{aligned} 5 &= 1 + 1 + 1 + 1 + 1 \\ &= 1 + 1 + 3 \\ &= 1 + 3 + 1 \\ &= 3 + 1 + 1 \\ &= 1 + 4 \\ &= 4 + 1. \end{aligned}$$

- **Subproblems** Let $f(n)$ be the number of ways in which n can be represented using the numbers 1, 3 and 4. Each state is represented by a single integer, n .
- **Recurrence** For $n \geq 4$, if we already know the answers for $f(n-1)$, $f(n-3)$ and $f(n-4)$, then the answer for $f(n)$ is given by
$$f(n) = f(n-1) + f(n-3) + f(n-4)$$
- **Base cases** By inspection, we can see that $f(0) = 1$, $f(1) = 1$, $f(2) = 1$ and $f(3) = 2$.

- **Complexity** Since we have $O(n)$ values of f to calculate, each taking $O(1)$ time to calculate, assuming that the subproblems it depends on have already been calculated, the algorithm has overall time complexity $O(n)$.

- **Implementation**

```
f[0] = 1, f[1] = 1, f[2] = 1, f[3] = 2;  
for (int i = 4; i <= n; i++) {  
    f[i] = f[i-1] + f[i-3] + f[i-4];  
}
```

- A neat trick allows us to optimise this implementation to use $O(1)$ memory without changing the time complexity.

```
f[0] = 1, f[1] = 1, f[2] = 1, f[3] = 2;  
for (int i = 4; i <= n; i++) {  
    f[i%4] = f[(i-1)%4] + f[(i-3)%4] + f[i%4];  
}
```


Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- There are two main ways of implementing (and thinking about) DP solutions.
- These are most commonly referred to as **top-down** (*memoised*) and **bottom-up**

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Top-down dynamic programming takes the mathematical recurrence, and translates it directly into code.
- Answers to subproblems are cached to avoid solving them more than once. Caching function return values is widely known as *memoisation*.
- Top-down implementations are usually the easiest, because this is how most people naturally think about DP solutions.

```
int f(int n) {  
    // base cases  
    if (n == 0 || n == 1) return 1;  
    // return the answer from the cache if we already have one  
    if (cache[n]) return cache[n];  
    // calculate the answer and store it in the cache  
    return cache[n] = f(n-1) + f(n-2);  
}
```

- **Warning:** if 0 is a valid answer to a subproblem, initialise your cache array to something that isn't a valid answer.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Bottom-up dynamic programming starts at the base cases and builds up all the answers one-by-one.

```
|| f[0] = 1, f[1] = 1;
|| for (int i = 2; i <= n; i++) { f[i] = f[i-1] + f[i-2]; }
```

- Warning:** when answering a subproblem, we must make sure that all subproblems it will look at are already answered.
 - In this example, the order in which states depend on each other is straightforward; this is not always the case.
 - In general, the dependency between DP states forms a **directed acyclic graph (DAG)**.
 - If the state dependency graph has a cycle, it's not a valid DP!
- Some algorithms are easier to think about this way. For example, the Floyd-Warshall algorithm is a DP, most easily implemented bottom-up.

- Top-down generally admits a more direct implementation after finding a recurrence.
 - It is more convenient on recursive structures like trees.
 - It only ever touches states that are necessary to compute, which can make it significantly faster for some problems.
- Bottom-up also has its own advantages.
 - Often, there are characteristics of the state space that allow for space optimisations only possible going bottom-up.
 - It also doesn't have the recursive overhead inherent to the top-down approach.
 - You usually more control, which is useful for more advanced techniques, e.g. speeding up the recurrence with a data structure.
- **Summary:** If you have a choice, pick your preference. However, for trees, we'll generally only do top down. And for many more advanced techniques, we'll probably only do bottom up.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Cool. Now we know what DP is (hopefully).
- The above helps you recognize and (hopefully) code a DP someone tells you.
- But all of this is not particularly useful for finding the right states or the right recurrence.
- What follows is a useful strategy to make these decisions, and some examples to demonstrate.
- But really, the only method that works for sure is practice.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- To get a gauge of whether a problem is DP and what the DP might look like, it's often easier to start with the recurrence rather than the state.
- We'd like to choose a state which contains the minimum amount of information while letting us figure out what steps are valid as we build up from trivial subproblems to the original problem.
- It's *not* expected that you'll pick the right state immediately. It's more a process of trial and error, *even if you are very experienced*.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- ① Choose some order to do the problem in. So essentially, how are you going to build up your solution?
 - Often implicit in the state, but a good starting point, since it strongly suggests what your state might be.
- ② Pick a tentative state. Initially, it should just contain the parameters necessary to determine the end result.
 - E.g: If you need to output the best answer assuming you take X items, then number of items should probably be in your state.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- ③ Test whether your state specification is sufficient by trying to make a recurrence.
 - Often you'll be unable to determine which moves are legal and which aren't, using only the information stored in your state.
 - This usually means that you need to add more information to the state. Add an extra parameter to address this, then repeat this step.
- ④ Repeat this until it stabilizes or you realise you should try something else.

- Now, you should at least have a DP algorithm.
- Be happy about this, you've probably just changed something from exponential to polynomial.
- Is it good enough though? **Calculate** your complexity.
- What if it isn't? A few directions to go from here.

- ① Look at your state. Is it bad? If so, can you fix this?
 - Is everything in our state necessary? Can we determine the valid moves from a subset of the state? More difficult: can we move anything around to improve the state?
 - Maybe our order to start with was incorrect.
- ② Look at your recurrence. Is there some nice structure to it? If so, it is likely a suitable data structure will speed it up.
 - In particular, any recurrences that are ranges should make you think of *range tree*

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP
- 5 DP with Data Structures
- 6 More example problems

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- All the DP problems we've seen so far have a simple, one-dimensional state.
- However, it is easy to extend DP to states of higher dimensions.
- The hardest part of finding a DP solution is usually identifying a state that makes sense for the problem, and more dimensions just add more possibilities.

- **Problem Statement** To further your academic career, you have decided to steal some textbooks from the library. Unfortunately the bag you have brought is far too small, and won't fit all of the books.
There are n books, the i th has a given size s_i and a value v_i (representing how valuable it is to you). Your bag has a given maximum capacity S : the sizes of all the books you take with you must total less or equal to this.
Security is coming, and you want to maximise the total value of the books you're taking. What is the maximum value you can fit in your bag?
- **Constraints** $1 \leq n \leq 5000$, $1 \leq S \leq 5000$.
 $1 \leq s_i, v_i \leq 5000$. All numbers are integers.

- We can't try every possible selection of books, because that would be $O(2^n)$ possibilities.
- We can start optimising by first observing that the order we put the books in the bag doesn't matter.
- In order to place a book in our bag, what information do we need to know?
 - ① The amount of space remaining in our bag
 - ② A guarantee that we haven't already put this book in our bag
- If we order the books by their given numbers, we have an ordering for free: if we are up to book i , then we've already considered books 1 through $i - 1$, and not books i through N .

- This suggests the state

(i, j)

where:

- i is the book we are currently considering,
 - we have already considered all the books before i , and
 - j is the amount of space used in the bag.
- We ask the question $f(i, j)$: how much value can I fit into j units of space, using only books from 1 through i ?
 - Then $f(n, S)$ will give the answer to the problem.
 - Can we find a recurrence that answers this question in terms of smaller ones?

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- $f(i, j)$: how much value can I fit into j units of space, using only books from 1 through i inclusive?
- If we consider only book i , we have two choices:
 - If we put book i in our bag, we will use s_i space and gain v_i value. Then the best value we could make would be $f(i - 1, j - s_i) + v_i$.
 - If we *don't* put book i in our bag, we will not use any space or gain any value. Then the best value we could make would be $f(i - 1, j)$.
- Thus we obtain the recurrence

$$f(i, j) = \max(f(i - 1, j - s_i) + v_i, f(i - 1, j)).$$

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- What if $j - s_i < 0$? To simplify the recurrence, we can simply include in our base cases that $f(i, j) = -\infty$ whenever $j < 0$, for all i . Then no solution that tries to use such an answer will ever be the best one.
- What about base cases that can actually result in successful answers?
- $f(i, 0) = 0$ for all i (no space to put books in).
- Also, $f(0, j) = 0$ for all $j \geq 0$ (no books to get value from).

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Complexity** Our state includes two parameters, one with n possibilities and the other with S possibilities, so there are a total of nS states.

Each state checks a constant number (at most 2) other states to obtain an answer, so each state takes $O(1)$ time to calculate.

Thus, the total time complexity of this algorithm is $O(nS)$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

• Top-Down Implementation

```
const int INF = 1000*1000*1000+7;
const int UNKNOWN = -1;

const int N = 5050, MAXS = 5050;
int cache[N][MAXS]; // initialise to -1 because 0 is a valid answer

int f(int i, int j) {
    // base cases
    if (j < 0) return -INF;
    if (i == 0 || j == 0) { return 0; }
    // check cache
    if (cache[i][j] != UNKNOWN) { return cache[i][j]; }
    // calculate answer
    return cache[i][j] = max(f(i-1, j-s[i]) + v[i], f(i-1, j));
}

int solve (int n, int S) {
    for (int i = 0; i <= n; i++) { fill (cache[i], cache[i]+S+1, UNKNOWN); }
    return f(n, S);
}
```

- This implementation reduces the need to bounds-check for the large number of base cases.

- In a bottom-up implementation, we are responsible for scheduling the computations ourselves.
 - We need an order that respects the dependencies: each state depends on i which are *less* so increasing order of i works.
- Also, we need to bounds-check carefully now, to make sure we don't read outside our array.
- **Bottom-Up Implementation**

```
const int N = 5050, MAXS = 5050;
int dp[N][MAXS]; // zero-initialised; correct for i = 0 and j = 0

for (int i = 1; i <= n; i++) {
    // everything with smaller i will be solved by the time we get here
    for (int j = 1; j <= S; j++) {
        int m = dp[i-1][j]; // skip book i
        // take book i; bounds check to skip when array index is negative
        if (j - s[i] >= 0) { m = max(m, dp[i-1][j-s[i]] + v[i]); }
        dp[i][j] = m;
    }
}
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP**
- 5 DP with Data Structures
- 6 More example problems

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Sometimes your state needs to be much bigger than the DPs you are used to.
- A common trick is to make your state a set.
- So your state space is $2^n \cdot (\text{extra metadata})$.
- Seems bad but still a lot better than $n!$ which is usually the alternative.
- Especially useful with NP-hard problems involving finding a permutation. TSP (Travelling Sales Person) is the most well-known such example.
- Practically, these problems can often be detected by having small bounds (e.g. $n \leq 20$).

- To represent our state, we don't store an actual set. This is unwieldy and probably slow.
- Instead we use a *bitset*.
- A bitset is an integer which represents a set. The i th least significant bit is 1 if the i th element is in the set, and 0 otherwise. For example the bitset 01101101 represents the set $\{0, 2, 3, 5, 6\}$.
- In this way, we can use an integer to index any subset of a set, amongst other things.
- This is much faster, especially if you use built-in bit operations to manipulate the set.

Useful operations for manipulating bitsets:

- Singleton set: $1 \ll i$
- Set complement: $\sim x$
- Set intersection: $x \& y$
- Set union: $x | y$
- Symmetric difference: $x \wedge y$
- Membership test: $x \& (1 \ll i)$
- Size of set
 - C++20, with `<bit>`: `popcount(x)`
 - before C++20, with GCC: `__builtin_popcount(x)`
- Least significant bit (or an arbitrary bit): $x \& (-x)$
- Iterate over all sets and subsets:

```
// for all sets
for (int set = 0; set < (1<<n); set++) {
    // for all subsets of that set
    for (int subset = set; subset; subset = (subset-1) & set) {
        // do something with the subset
    }
}
```


Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Problem statement** You want to tile your roof with n tiles in a straight line, each of which is either black or white. Due to regulations, for every m consecutive tiles on your roof, at least k of them must be black. Given n , m and k ($1 \leq n \leq 60$, $1 \leq k \leq m \leq 15$, $m \leq n$), how many valid tilings are there?
- **Example** If $n = 2$, $m = 2$ and $k = 1$, there are 3 different tilings: BB, BW, or WB.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- A counting problem, a bit different from what we might be used to. Our DP will associate with a state how many configurations correspond to that state. The rest (for now) will mostly be the same.
- Let's start with an obvious state. How about the number of valid tilings with n tiles. Then hopefully we can step to $n + 1$ by laying one more tile.
- Then our step is to lay either a black tile or white tile.
- But how do we know whether we can lay a white tile?
- We need to know if there are only $k - 1$ black tiles in the last $m - 1$ tiles. If so, the next tile must be black.
- We should add that to our state.

- Now our state is n , the number of tiles laid and b , the number of black tiles that are in the last $m - 1$ tiles.
- We can now tell whether the $n + 1$ -th tile is forced to be black.
- Let $dp[n][b]$ be the number of tilings of n tiles where b of the last $m - 1$ are black.
- Then $dp[n][k - 1]$ can be counted once (tile $n + 1$ is black only), and each $dp[n][j]$ ($k \leq j < m$) can be counted twice (tile $n + 1$ can be either colour).
- Counted towards what though? This is finding $dp[n + 1]$, but we'll need all the $dp[n + 1][b]$ values in the next round, and we haven't found those.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- There's a fundamental problem: we only store how many tiles in the last $m - 1$ are black.
- But say we add a black tile to the right. Then how do we know if $\#(\text{black tiles in the last } m)$ increases or stays the same?
- We need to know what the $(m - 1)$ th tile was.
- And because of this, in the next round we will need to know what the $(m - 2)$ th tile was, and so on.
- So we should amend our state to include *which* of the last $m - 1$ tiles are black, not just *how many*.
 - Implementation is simpler if we just store the last m

- Let $f(i, S)$ be the number of ways to have tiled the first i tiles, such that out of the last m tiles, the ones that are black are exactly the ones in the set S .
- For the recurrence, we can either set the new tile to be black or white. Reflecting this in our state is just applying the right bit operations from earlier.

- Recurrence**

$$f(i, S) = f(i-1, S \gg 1) + f(i-1, (S \gg 1) | (1 \ll (m-1))),$$

where $|S| \geq k$, or 0 otherwise. Tile $(i-m)$ is white in the first term and black in the second.

- Base case** We have $f(m, S) = 1$ iff $|S| \geq k$. We don't need to consider $f(i, S)$ for any $i < m$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Complexity** There are $O(n2^m)$ total states to calculate, and each state takes $O(1)$ to compute, so this algorithm runs in $O(n2^m)$ time.

Implementation

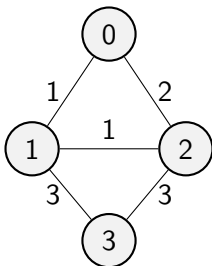
```
for (int set = 0; set < (1<<m); set++) {
    dp[m][set] = (popcount(set) >= k); // base case
}

for (int i = m+1; i <= n; i++) {
    fill(dp[i], dp[i] + (1<<m), 0);
    for (int set = 0; set < (1<<m); set++) if (popcount(set) >= k) {
        dp[i][set] = dp[i-1][set>>1]
                    + dp[i-1][(set>>1)|(1<<(m-1))];
    }
}

long long ans = 0;
for (int set = 0; set < (1<<m); set++) { ans += dp[n][set]; }
```

- **Space Optimisation** We could also exploit the fact that the answer for $f(i, S)$ only ever relies on the answers for $f(i-1, S')$ to use only $O(2^m)$ memory.
 - Simply replace all instances of $dp[i][j]$ with $dp[i\%2][j]$ and so on.
- **Moral** The brute force has $2^n \approx 10^{18}$ possibilities, but we can aggregate tilings into only $2^m \leq 32,768$ groups based on how the last m tiles are placed.
 - We don't care precisely how the tiles were placed long ago; only the last m can impact any future tile placements.

- **Problem Statement:** Given a weighted, bidirectional graph with n nodes, find the length of the shortest path starting from node 0 that visits every node exactly once.



- **Sample Input:**
- **Sample Output:** 5, path is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Let us try the problem in a natural order: increasing number of nodes on a path.
- So our state is (#nodes in path) and we store the shortest path with that many nodes.
- Then our recursion is ...whoops, we need to know where we are as well.
- So let us amend the state, our state is (#nodes in path, last node in path).
- Then our recursion is to try every next node.
- But how do we know if we have visited a node twice?
- We have to keep this in our state ...

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- So our state is (S, e) . We store the shortest path that starts at 0, ends at e and uses the nodes in S exactly once. We will denote this $f(S, e)$.
- Now we know what next moves we can make. So we have a hope of forming a recurrence.
- We could try considering all possible cities we could have came from:

$$f(S, e) = \min_{p \in S \setminus \{e\}} f(S \setminus \{e\}, p) + \text{dist}[p][e]$$

- Alternatively, it's more natural to push this forward and think of the next cities we can go to.
- Exercise: Translate the above into bit operations.

Example: Travelling Sales Person

51

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

```
#include <algorithm>
using namespace std;

const int N = 20;
const int INF = 1e9;
int n, adj[N][N]; // assume this is given.
int dp[1<<N][N]; // dp[x][i] is the shortest 0->i path visiting set bits of x

int tsp (void) {
    for (int mask = 0; mask < (1<<n); mask++)
        for (int city = 0; city < n; city++) { dp[mask][city] = INF; }
    dp[1][0] = 0; // 1 represents seen set {0}

    int ans = INF;
    for (int mask = 1; mask < (1<<n); mask++) // for every subset of cities seen
        so far
        for (int cur = 0; cur < n; cur++) {
            if (mask & (1<<cur)) { // cur must be one of the cities seen so far
                int cdp = dp[mask][cur]; // distance travelled so far
                if (mask == (1<<n) - 1) { ans = min(ans, cdp); } // seen all cities
                // note we don't have to add adj[cur][0] to return to vertex 0
                for (int nxt = 0; nxt < n; nxt++)
                    if (!(mask & (1<<nxt))) { // try going to a new city
                        // new seen set is mask union {nxt}, and we will be at nxt
                        // distance incurred to get to this state is now no worse than
                        // cdp (current distance incurred) + edge from cur to nxt
                        dp[mask|(1<<nxt)][nxt] = min(dp[mask|(1<<nxt)][nxt], cdp + adj[cur][nxt]);
                    }
            }
        }
}
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP
- 5 DP with Data Structures**
- 6 More example problems

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Sometimes you have the right state space but the cost of recursion is too high.
- In such cases, examine the structure of the recurrence closely.
- Often it can be sped up with the right choice of data structure.
- One common example is when your recursion naturally involves a *range* query.

- You have already seen one example of this: LIS.
- The recurrence we end up with by going left to right is
$$\text{best}[i] = 1 + \max \text{bestWithEnd}[0, a[i]].$$
- Note in LIS, it was not clear from the outset that a range tree would play a role. You had to write down the recurrence to see it.

- **Problem statement** You have n fireworks available to you. Each firework has a position x_i on the horizon, a *combo range* $[l_i, r_i]$ and a score s_i .

You can enable or disable each firework. Enabled fireworks will be launched and contribute their score to the total. However, a firework can only be launched if its combo range includes the position of the *most recent* firework to be launched.

The fireworks are specified in order of firing time. You *cannot* change this order.

What is the maximum total score you can obtain?

- **Constraints** $1 \leq n \leq 100,000$, $1 \leq x_i \leq 500,000$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- For example, suppose we had three fireworks located at positions 1, 2 and 3 in that order, and that their combo ranges were $[1, 5]$, $[3, 5]$ and $[0, 1]$ respectively.
- We can make a fireworks display with any individual firework, because there would be no previous firework restricting the combo range.
- However, we could never launch the second firework with any of the others.
 - The only firework before it (the first one) lies outside its combo range.
 - On the other hand, its position is not within the combo range of the only firework after it (the third one).

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Due to the firing order specified in the problem, we have a natural order to start with.
- Then the obvious state to start with is just the last firework we've launched. For each firework, we store the highest obtainable total with a chain ending at that firework.
- Suppose that for each previous firework, we have the best total from a chain of zero or more fireworks leading up to it. Now we want to pick the best eligible previous firework.
- 'Eligible' here enforces that the previous firework has to be inside our combo range.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

• Top-Down Implementation

```
int f(int i) {
    if (i == 0) return 0;
    if (cache[i]) return cache[i];
    int m = s[i];
    for (int j = 1; j < i; j++) {
        if (x[j] >= l[i] && x[j] <= r[i]) {
            m = max(m, f(j) + s[i]);
        }
    }
    return cache[i] = m;
}
```

• Bottom-Up Implementation

```
ll res = 0;

for (int i = 1; i <= n; i++) {
    dp[i] = s[i];
    for (int j = 1; j < i; j++) {
        // try the jth as the penultimate firework
        if (x[j] >= l[i] && x[j] <= r[i]) {
            dp[i] = max(dp[i], dp[j] + s[i]);
        }
    }
    // update answer
    res = max(res, dp[i]);
}
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- We have n states to calculate, and each one takes $O(n)$ time.
- This is an $O(n^2)$ algorithm, and with n up to 100,000, this is not going to run in time.
- It seems unlikely that we can change the state space to be anything other than linear, but the recurrence looks simple enough.
- What is the actual recurrence?

- **Recurrence** Let $f(i)$ be the best score possible ending at the i th firework (in the given order). Then

$$f(i) = s_i + \max\{f(j) \mid j < i, l_j \leq x_i \leq r_j\},$$

where the maximum is zero if no such j exists.

- This is a *range* constraint.
- So we should be able to query this maximum using a range tree, and obtain a better solution!
- There are still n states, but now each one takes only $O(\log n)$ time to calculate, so we obtain a solution in $O(n \log n)$ time.

• Bottom-Up Implementation

```
int query(int a, int b); // max query range tree of size 500,000
int update(int a, int v); // update index a to value v

ll res = 0;
for (int i = 1; i <= n; i++) {
    // calculate best score ending in i-th firework using the range tree
    dp[i] = query(l[i], r[i] + 1) + s[i];
    // add i-th firework to RMQ
    update(x[i], dp[i]);
    // update final answer if necessary
    res = max(res, dp[i]);
}
```

- A top-down implementation is not as easy to come up with in this case. Why?

- **Problem Statement:** Consider the $n + 1$ points on the real line $0, 1, \dots, n$. You are given m segments, each with a range $[s_i, e_i]$ and a cost c_i .

Output the minimum cost necessary to obtain a subset of the segments which covers all $n + 1$ points.

- **Input Format:** First line, n, m . $1 \leq n, m \leq 100,000$. The following m lines describe a segment as a triple (s_i, e_i, c_i) .

- **Sample Input:**

5 4
0 5 10
0 3 4
2 5 4
0 4 5

- **Sample Output:**

8

- **Explanation:** Take the second and third segments.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- What order should we attempt the problem in?
- Let's try just processing the segments one by one.
- What is the state?
- To answer the question, we need to store what indices we have covered.
- Okay, that's a bit excessive.
- **Note:** There was nothing special about the order of the segments, we essentially processed the segments in an arbitrary order.
- It is generally better to at least have some meaningful order.

- Alternatively, we can focus on the array.
- Then a natural order is left to right.
- But what would our state be then? This is less obvious.
- A first guess would be i , where we are up to, and S which indices to the left of i are covered.
- But this just gets us back to where we started.
- The **key** here is to make the state just i . Then we denote $dp[i]$ to be the min cost of segments to cover *exactly* $[0, i]$.

- A very reasonable question is, is this enough to form a recursion?
- Let's just try!
- To go from i to $i + 1$ what do we need to do?
- We need to cover index $i + 1$. What does this mean for our set of segments?
- We need to pick a segment ending at $i + 1$! (Note, we are defining $dp[i]$ to be the min cost to cover *exactly* $[0, i]$. So we ignore segments ending past $i + 1$)
- So we now have our choices. How do we form the recurrence?

- Let's say we pick segment $[s, i + 1]$ with cost c . What do the rest of our segments have to satisfy?
- **Answer:** They must cover a range $[0, e]$ where $e \geq s - 1$.
- So assuming we pick this segment, we have

$$dp[i + 1] = c + \min_{j \in [s-1, i]} dp[j]$$

- Complexity appears to be $O(n^2 m)$: for each point $i + 1$, for each segment, calculate this min.
 - Far too slow.
 - State space is probably optimal already, so try to speed up the recurrence.

- Recurrence:

$$dp[i + 1] = c + \min_{j \in [s-1, i]} dp[j]$$

- **Observation:** We don't have to try all segments, just the ones ending at $i + 1$.
- Preprocessing segments by endpoint reduces the time complexity to $O(nm)$:
 - For each point $i + 1$, for each segment *ending here*, calculate the min.
 - The min is only calculated once per segment.
- Getting closer, but still too slow.

- Recurrence:

$$dp[i+1] = c + \min_{j \in [s-1, i]} dp[j]$$

- **Observation:** This min is another *range* query!
- So we would hope we can support this with a range tree!
- Range min tree
 - Like LIS, range tree over *values* $dp[j]$ not indices j
 - Default value should 'lose' to any other value, so pick ∞ rather than 0.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

Implementation

```
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>
using namespace std;

typedef long long ll;
const int N = 100100;
const ll INF = (1LL << 61);
int n, m;

vector<pair<int, ll>> segments[N]; // (start, cost)
ll dp[N];
int tree[1<<18]; // range min tree with point update
void update(int p, ll v, int i = 1, int cL = 0, int cR = n);
ll query(int qL, int qR, int i = 1, int cL = 0, int cR = n); // [qL, qR)

int main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int s, e, c;
        cin >> s >> e >> c;
        segments[e].emplace_back(s, c); // preprocess: collate by end
        point
    }

    for (int i = 0; i <= n; i++) { update(i, INF); }

    // to be continued
```

• Implementation (continued)

```
for (int i = 0; i <= n; i++) {  
    dp[i] = INF;  
    for (auto seg : segments[i]) {  
        ll prevcost = seg.first == 0 ? 0 : query(seg.first-1, i);  
        dp[i] = min(dp[i], prevcost + seg.second);  
    }  
    update(i, dp[i]);  
}  
cout << dp[n] << '\n';  
}
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Complexity? $O((n + m) \log n)$: make m range queries and n point updates to a range tree of size $O(n)$
- There are many problems along these lines, of doing DP on a line with choices given by intervals. It shouldn't be a surprise many involve range trees.
- A key was to be clear what the state represented.
- **Exercise:** An alternative is to make $dp[i]$ the min cost for covering **at least** $[0, i]$. Work out the details for this approach.
- **Exercise:** What changes if we require the intervals to cover the entire interval $[0, n]$ not just the integer parts?

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- 1 Reminder: Algorithmic Complexity
- 2 What is dynamic programming?
- 3 2D DP
- 4 Exponential DP
- 5 DP with Data Structures
- 6 More example problems

- **Problem Statement:** You have a $H \times W$ block of marble, divided into 1×1 cells. Each cell has a value. You want to pick a subset of cells to make a building. The restrictions are:
 - Each cell of the building must lie on either the ground or another cell of the building.
 - The cells chosen for the building for each of the H rows must be contiguous.

What is the maximum possible sum of values of a valid building?

- **Input:** First line 2 integers, H, W . $1 \leq H, W, \leq 1000$. Next H lines each have W integers, the value of the cells. These values can be negative.
- **Source:** Australian Informatics Invitational Olympiad 2014.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Sample Input:**

```
3 4
-9 1 -9 1
1 1 -9 1
1 1 1 1
```

- **Sample Output:**

7

- **Explanation:** Pick all 4 cells in the bottom row, the left 2 in the second row and just the 2nd in the third row.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- This is less obviously DP. Let's just pick something to look at and start from there.
- One natural starting point is to consider the problem cell by cell.
- What order? Let's say bottom to top, and left to right within each row.
- So our state is cell we are considering.
- And our choice needs to be whether to put the next cell into the building.
- But how do we know if we can include the next cell?

- Okay, our state is too small, what do we need to add?
 - What cells on the previous row are in our building.
 - Something about what cells in our current row are in our building. **Why?**
 - Actually we need to store exactly what cells in our current row are in our building.
- What is the size of our state space now?
- Depends a bit on the implementation. Maybe something like $O(WH \cdot W^2 \cdot W^2)$.
- This is a problem. We can optimize the recurrence but if our state space is too large we are just dead in the water.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Let's try a different state.
- We can note that each row is very structured, perhaps we can do a row at a time.
- Instead of building cell by cell, let's build row by row.
- So instead of having $O(1)$ moves, our moves are "pick a contiguous selection of cells on this row and put it in our building".
- What is our state? How do we know if a move is valid?

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- We need to know if the segment we have chosen lies within the segment of the building on the previous row.
- So our state should be what row we are up to and what cells in the previous row are in the building.
- How big is our state?
- $O(H \cdot W^2)$.
- Still a bit too large but this is progress. It is possible to speed up the recurrence to $O(1)$ so we are nearly there.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- If you were solving the problem yourself, you would probably keep trying this direction for a while.
- But ultimately, it seems this is the limit of how small we can make the state space.
- Each of the parameters is necessary.
- We probably need to pick a different direction for our DP.
- Let's go back to the drawing board.
- **Draw things!**

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Let's try going left to right. So we are now making our building column by column.
- What are our moves? Same as before, let's try picking the cells for each column.
- What is our state? How do we know a move is valid? Boils down to, what are the constraints for a valid building in terms of its columns?
- **Key Observation:** It is ternary! The building increases height up to a point then decreases in height.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Key Observation:** It is ternary! The building increases height up to a point then decreases in height.
- Okay, let us start with the obvious state, which column we are up to.
- Is this enough to know which moves we can make?
- No. We don't even know if the building increased in height from the previous column.
- What do we need to store to know this?
- Okay, our state is now
(current column, height of current column).

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Key Observation:** It is ternary! The building increases height up to a point then decreases in height.
- State:
(current column, height of current column).
- Is this enough?
- Note: It is always okay to decrease in height. But if our move increases the height, how do we know if it is a valid move?
- No. Not enough. We need to know if at least once in the past our building has decreased in height.
- New state:
(current column, height of current column, has the height of the building ever decreased?)

- **Key Observation:** It is ternary! The building increases height up to a point then decreases in height.
- State:
(current column, height of current column, has the height of the building ever decreased?)
- Is this enough?
- It is enough to tell us what moves are valid in the current column.
- What is the state space?
- $O(WH)$.
- So we should be optimistic. So we should try to define a recurrence.

- State:
(current column, height of current column,
has the height of the building ever decreased?)
- Let $f(j, h, b)$ be the best building using the first j columns,
with h cells picked in column $j-1$ and $b = 1$ iff the height
of the building has decreased at some stage.
- Recurrence? Try $b = 0$ and $b = 1$ separately.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- First try $b = 0$.
- Previous column must be the same height or shorter, and can't have had a prior decrease in height.

- Recurrence is

$$f(j, h, 0) = \max_{h' \leq h} f(j-1, h', 0) + \sum_{i \geq H-h} a[i][j-1].$$

- Complexity?
- $O(WH)$ state space, $O(H)$ recurrence. Overall $O(WH^2)$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Now that the state space is good, we can hope to speed up the recurrence.

- Recall

$$f(j, h, 0) = \max_{h' \leq h} f(j-1, h', 0) + \sum_{i \geq H-h} a[i][j-1].$$

- This looks relatively structured. **Why?**
- Everything is a range!
- The first part is a range max. We know how to do these.
- The second part is a range sum of input. We can precompute this. **How?**
- So we should be able to do $O(WH \log H)$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- We can do better.
- If we fix both j and h , it takes $O(H)$ time to compute the max in the first term and the sum in the second term of $f(w, h, 0)$.
- However, if we fix only j , and consider $h = 0, 1, \dots, H$ together, we see that each max takes only $O(1)$. **Why?**

- Compare two similar terms:

$$f(j, h-1, 0) = \underbrace{\max_{h' \leq h-1} f(j-1, h', 0)}_M + \underbrace{\sum_{i \geq H-h+1} a[i][j-1]}_S$$

$$f(j, h, 0) = \underbrace{\max_{h' \leq h} f(j-1, h', 0)}_{M'} + \underbrace{\sum_{i \geq H-h} a[i][j-1]}_{S'}$$

- Do we actually need to calculate M' and S' from scratch?
- No! We can do each in constant time:

$$M' = \max(M, f(j-1, h, 0)) \text{ and } S' = S + a[H-h][j-1].$$

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Next try $b = 1$.
- Previous column must be the same height or taller, but *could* have had a prior decrease in height *or not*.

- Recurrence is

$$f(j, h, 1) = \max_{h' \geq h} f(j-1, h', \cdot) + \sum_{i \geq H-h} a[i][j-1].$$

- Again, we aim to compute this in $O(1)$ rather than $O(H)$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- The max includes extra terms as h decreases.
- Compare two similar terms:

$$f(j, h+1, 1) = \underbrace{\max_{h' \geq h+1} f(j-1, h', \cdot)}_M + \underbrace{\sum_{i \geq H-h-1} a[i][j-1]}_S$$

$$f(j, h, 1) = \underbrace{\max_{h' \geq h} f(j-1, h', \cdot)}_{M'} + \underbrace{\sum_{i \geq H-h} a[i][j-1]}_{S'}$$

- First compute $f(j, H, 1)$ in full, in $O(H)$ time.
- Then the rest take $O(1)$ each:

$$M' = \max(M, f(j-1, h, 0), f(j-1, h, 1))$$

$$S' = S - a[H-h-1][j-1].$$

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Recap. Our state: $f(j, h, b)$.

- Recurrences:

$$f(j, h, 0) = \max_{h' \leq h} f(j-1, h', 0) + \sum_{i \geq H-h} a[i][j-1]$$

$$f(j, h, 1) = \max_{h' \geq h} f(j-1, h', \cdot) + \sum_{i \geq H-h} a[i][j-1].$$

- We speed up the RHS calculations by doing them iteratively per column.
- $O(WH)$ states with recursion cost $O(H)$ per column, hence overall recursion cost $O(WH)$.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

```
#include <algorithm>
#include <iostream>
using namespace std;

typedef long long ll;

const int N = 1010;
ll a[N][N]; // (i,j)
ll dp[N][N][2]; // (j,h,b)

int main (void) {
    int H, W;
    cin >> H >> W;
    for (int i = 0; i < H; i++) {
        for (int j = 0; j < W; j++) {
            cin >> a[i][j];
        }
    }
}
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

```

11 M, S;
for (int j = 1; j <= W; j++) {
    // b = 0 case
    M = 0; S = 0;
    dp[j][0][0] = M + S;
    for (int h = 1; h <= H; h++) {
        M = max(M, dp[j-1][h][0]);
        S += a[H-h][j-1];
        dp[j][h][0] = M + S;
    }
    // b = 1 case
    M = 0; S = 0;
    for (int i = 0; i < H; i++) { S += a[i][j]; }
    dp[j][H][1] = M + S;
    for (int h = H-1; h >= 0; h--) {
        M = max(M, dp[j-1][h][0], dp[j-1][h][1]);
        S -= a[H-h-1][j-1];
        dp[j][h][1] = M + S;
    }
}

11 ans = 0;
for (int h = 0; h <= H; h++) {
    ans = max(ans, dp[W][h][0]);
    ans = max(ans, dp[W][h][1]);
}
cout << ans << '\n';
}
    
```

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

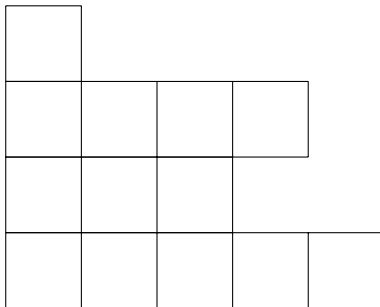
Exponential
DP

DP with Data
Structures

More example
problems

- Let's summarize.
- The problem seems difficult and convoluted. But the solution is quite natural, except looking left to right and characterizing the condition for going left to right.
- Even for that part, it is always worth making examples and trying different orders.
- Our choice of state space was (mostly) dictated just by the order we picked.
- Our recurrences followed through by translating our requirement.
- Speeding up the recurrence is natural from looking at the formula.
- **Moral (hopefully):** None of this is magic. Almost all of it is fairly logical.

- **Problem statement** You have a set of n ($1 \leq n \leq 1000$) rows from a $n \times n$ chessboard, with some of the squares cut out from the right. How many ways are there to place k rooks on this chessboard without any rook threatening any other rook, modulo $10^9 + 7$?



Dynamic Programming

Reminder:
Algorithmic
Complexity

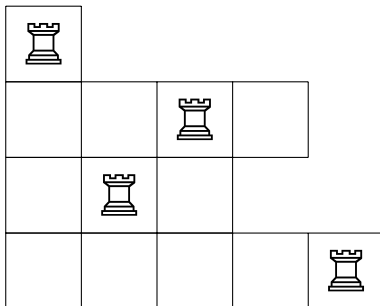
What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems



$n = 4, k = 4$: 8 ways

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Let us pick an order.
- Top row to bottom row seems natural.
- Then our state is which row we are up to, and we will store the number of ways to place rooks up to this row.
- Our moves are to try all places we can put a rook on the current row.
- How do we know if a move is valid?

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- In other words, how do we know which columns are free?
- We don't know this. So we need to keep the set of free columns in our state.
- We can use a bitset like before.
- How large is the state space?
- About $O(2^m \cdot n)$, not good enough.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- But we are counting things. Can we instead just store the number of rooks we've placed. Then hopefully the number of ways we can place a rook on our new row is $(\text{length of row}) - (\text{\#rooks placed})$.
- Alas, we don't know whether the rooks we've placed are in a column we can place to or not.
- Can we fix this?
- **Key Observation:** There was nothing special about the ordering of rows in this problem. Usually the order is suggested by the problem but in this case, there is no reason to have the rows in the order they are in.

Dynamic Programming

Reminder:
Algorithmic
Complexity

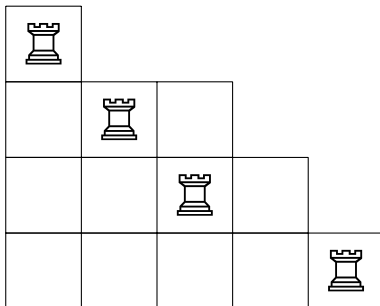
What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems



$n = 4, k = 4$: 8 ways

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Let's sort the sequence, so that the size of each row is non-decreasing
- Now, we know that if we place a rook on a row, we know that we can assume that every previously placed rook is in a cell that our row covers

Dynamic Programming

Reminder:
Algorithmic
Complexity

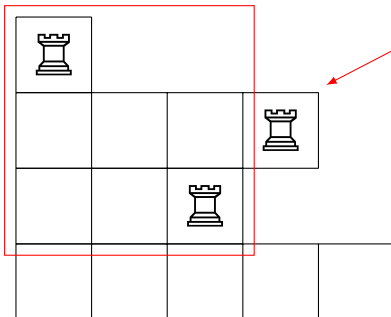
What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems



This never happens!

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Now, we can say that any rooks already placed will be either to the left or directly above us
- We can then formulate a recurrence that only needs to know about the current row and the number of rooks
- If we're on a row i of length l_i , then for every configuration of the rows above with j rooks already placed, we can place a rook for this row in $(l_i - j)$ places

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- **Subproblems** Let $f(i, j)$ be the number of ways to place j rooks on the first i rows, sorted by length.

- **Recurrence**

$$f(i, j) = f(i - 1, j) + f(i - 1, j - 1) \times (l_i - (j - 1))$$

- **Base case** The number of ways to place 0 rooks on 0 rows is 1 (*not* 0).

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

```
#include <algorithm>
#include <iostream>
using namespace std;

typedef long long ll;

const int N = 1010;
int l[N];
ll dp[N][N];
const ll MOD = 1000*1000*1000+7;

int main (void) {
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) { cin >> l[i]; }
    sort(l, l+n);

    dp[0][0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= k; j++) {
            dp[i][j] = dp[i-1][j]; // place no rooks in this row
            if (j > 0) { // or place a rook in this row
                dp[i][j] += dp[i-1][j-1] * (l[i-1] - (j-1));
                dp[i][j] %= MOD;
            }
        }

    cout << dp[n][k] << endl;
}
```

- This time around, the magic was in reordering the rows.
- This **is** magic. But this general idea is very useful. If you aren't given an order, make an order. It can't be worse than having no order.
- You see a similar idea in 0-1 knapsack. To avoid storing a bitset of used items, we just pick any arbitrary order to process the items in.

Dynamic Programming

Reminder:
Algorithmic
Complexity

What is
dynamic
programming?

2D DP

Exponential
DP

DP with Data
Structures

More example
problems

- Another path to a solution is to do the problem in column order.
- But you have to do it from right to left.
- Then you get the same property as this solution, without sorting.
- Note: column order isn't symmetrical as all rows are required to start from column 0.